JULIUS · DEV WORKFLOW

Workflow com Pull Requests

Como estruturar o desenvolvimento em branches e integrar mudanças de forma controlada.

1 O Problema com commits directos na main

Actualmente, cada mudança vai directamente para a **main**. Isso funciona para começar, mas cria problemas à medida que o projecto cresce:

Numa sexta à noite adicionas uma feature nova. Mas ela quebra outra coisa. Tens de reverter — e podes apagar trabalho que estava junto no mesmo commit. Ou tens de empurrar um hotfix por cima de código incompleto.

O workflow com PRs isola cada mudança na sua própria **branch**. A *main* só recebe código que já foi revisto e validado.

2 O Modelo de Branches

```
main ← sempre estável, vai para produção

  feat/ultimo-mes-filter ← nova funcionalidade

  fix/dashboard-chart-bug ← correcção de bug

  feat/budget-alerts ← outra feature em paralelo
```

💡 Cada branch existe isolada. Podes trabalhar em **feat/budget-alerts** durante dias sem que esse código incompleto afecte a **main**. Só quando está pronto é que se faz merge.

3 O Ciclo Completo — Passo a Passo

1 Partir sempre de uma main actualizada

Antes de começar qualquer trabalho, garantir que tens a versão mais recente.

```
git checkout main
git pull
```

2 Criar uma branch para a tarefa

Uma branch por feature ou bug fix. Nome descritivo com prefixo.

```
git checkout -b feat/budget-alerts
# ou
git checkout -b fix/extrato-filter-bug
```

3 Trabalhar e commitar normalmente

Commits pequenos e frequentes. Nesta fase não há pressão — o código ainda não está na main.

```
# editar ficheiros...
git add components/dashboard/BudgetAlert.tsx
git commit -m "Add budget alert component"

# mais trabalho...
git commit -m "Wire alert to useStats hook"
```

4 Fazer push e abrir o Pull Request

Enviar a branch para o GitHub e criar o PR com título e descrição.

```
git push -u origin feat/budget-alerts

# abrir PR via GitHub CLI
gh pr create \
  --title "feat: budget alerts para limites de categoria" \
  --body "Adiciona notificação quando o gasto mensal
ultrapassa o limite definido nas settings."
```

5 Rever o PR (auto-revisão)

Abrir o PR no GitHub e ler o diff com calma antes de mergear. É aqui que se apanham erros que passam despercebidos durante o desenvolvimento.

```
# ver o PR no browser
gh pr view --web
```

```
# ou ver o diff no terminal
gh pr diff
```

6

Fazer merge (squash)

O **squash merge** colapsa todos os commits de trabalho num único commit limpo na main. O histórico fica linear e semântico.

```
gh pr merge --squash

# actualizar a main local e apagar a branch
git checkout main
git pull
git branch -d feat/budget-alerts
```

4 Squash Merge — Porquê?

SEM SQUASH — HISTÓRICO NA MAIN

```
a1b2c3 WIP budget alerts
d4e5f6 fix typo
g7h8i9 more wip
j1k2l3 ok this works
m4n5o6 final tweaks
```

COM SQUASH — HISTÓRICO NA MAIN

```
p7q8r9 feat: budget alerts
        para limites de categoria
```

✦ Com squash, o **git log** da main conta a história do produto — uma linha por feature ou fix. Fácil de ler, fácil de reverter se necessário (`git revert p7q8r9`).

5 Commits directos vs. Pull Requests

SITUAÇÃO	SEM PRS	COM PRS
Feature incompleta	Fica na main, pode ir para produção	Fica na branch, invisível para a main
Bug introduzido	Misturado com outros commits, difícil isolar	Reverter 1 PR sem afectar o resto
Contexto da mudança	Só no commit message	Título + descrição + screenshots no PR

SITUAÇÃO	SEM PRS	COM PRS
Trabalho em paralelo	Commits conflituam na main	Cada branch isolada, merge quando pronto
Histórico do projecto	Commits de WIP poluem o log	Um commit limpo por feature (squash)
Revisão antes de mergear	Não existe	Leitura do diff completo antes do merge

6 Nomenclatura de Branches

`feat/` nova funcionalidade

`fix/` correcção de bug

`chore/` dependências, config, infra

```
# Exemplos reais para o Julius
feat/budget-alerts
feat/recurring-transactions
fix/dashboard-timezone-bug
fix/extrato-csv-encoding
chore/upgrade-nextjs-15
chore/supabase-types-regen
```

7 Uma Sessão de Desenvolvimento Típica

Início — **git pull + criar branch**
Partir de uma main limpa. Criar `feat/recurring-transactions`.

Durante — **Commits frequentes na branch**
Sem pressão. WIP, experiências, reverter — tudo fica na branch.

Pronto — **Push + gh pr create**
Abrir o PR. Escrever o que foi feito e porquê no body.

Revisão — **Ler o diff no GitHub**

Ver o PR como se fosse código de outra pessoa. Apanhar erros óbvios antes de mergear.

Merge

gh pr merge --squash

Um commit limpo entra na main. Branch apagada. Ciclo completo.

8 Integração com Claude Code

O Claude Code consegue gerir todo o workflow por ti. Em vez de correr os comandos manualmente, podes dizer:

```
# No início de uma sessão:
"cria uma branch feat/budget-alerts e implementa..."

# No final:
"abre um PR com o que fizemos hoje"

# Para uma correcção urgente:
"cria um fix/extrato-crash, corrige o bug e abre PR"
```

⚡ O Claude cria a branch, faz os commits com mensagens semânticas, faz push e abre o PR com título e descrição já preenchidos — incluindo um test plan.

Julius · Workflow Guide · Abril 2026